Python – Complete Lecture Notes

From Core Syntax to OOP, Functional Programming & Concurrency

> *"Python is designed so that any developer can learn it.*
> *It is meant for everyone, not just specialists."*
> — Guido van Rossum, Creator of Python (1991)

Topics Covered

- Why Python? Language overview
- Data types & memory model
- Operators & expressions
- Strings & collections
- Control flow & loops
- Functions & closures
- Modules & packages
- OOP – Classes & objects
- Inheritance & polymorphism
- Abstraction & decorators
- Exception handling
- Threads & multiprocessing

Contents

1. Python at a Glance

1.1 What is Python?

Python is a **high-level, general-purpose, multi-paradigm** programming language first released in **1991** by Guido van Rossum. "High-level" means it reads close to English – you don't write ones and zeros or assembly instructions. The interpreter converts your code to machine code at runtime.

Note

Why "high-level" matters: Assembly languages were hardware-close but painful. C/C++ improved things but are complex. Python's goal was simplicity – anyone should be able to program, not just specialists.

1.2 Multi-Paradigm Support

Python doesn't force one style. You can write code in whichever style fits your problem:

Paradigm	Core Idea	Examples in other languages
Procedural	Step-by-step instructions, functions	C
Object-Oriented	Model reality as objects	Java, C#
Functional	Functions as values, avoid state	Haskell, JS
Structured	Blocks, conditions, loops	Most languages

1.3 Why Python is Famous

- **Easiest to read/write:** Syntax feels like English.
- **Huge community:** Millions of developers = tons of libraries.
- **AI/ML ecosystem:** NumPy, Pandas, TensorFlow, PyTorch – all Python.
- **Web development:** Django, Flask, Tornado.
- **System scripting, automation, data science, GUIs.**

1.4 How Python Runs

1. You write `.py` files.
2. Python's **CPython interpreter** compiles to bytecode (`.pyc`).
3. The Python Virtual Machine (PVM) executes bytecode.
4. OS and hardware execute the final machine instructions.

Tip

You can test code in the Python REPL (interactive prompt) by typing `python` in your terminal. The >> prompt means you're in REPL mode. Type `exit()` to leave.

2. Variables, Memory Model & the Object System

2.1 Variables are References, Not Boxes

This is the single most important mental model shift from C/Java. In Python, a variable is not a container holding a value. It is a **label** (reference) that points to an **object** in memory.

```
a = 5          # Creates int object 5 in memory. 'a' points to it.
```

```
b = 5          # Python finds existing 5 object. 'b' ALSO points to same object.
print(id(a) == id(b))    # True -- same identity!

b = 6          # New int object 6 created. 'b' now points to 6.
print(id(a) == id(b))    # False -- different objects now.
```

Note

id() returns a unique integer identifying an object's memory location. Two objects with the same id are the same object.

2.2 Garbage Collection

When no variable points to an object, Python's garbage collector reclaims its memory. You don't manage memory manually.

```
a = 5     # object 5, referenced by a
b = 5     # same object 5, referenced by a and b
b = 6     # object 6 for b. Object 5 still has 'a'.
a = 10    # Now object 5 has ZERO references -> garbage collected.
```

2.3 String Interning

Python caches small integers (typically −5 to 256) and short strings as an optimization. This means `a = 5; b = 5` share the same object. For large strings or numbers, Python creates separate objects even with identical values.

```
a = "hello"
b = "hello"
print(id(a) == id(b))    # Usually True -- interned (short, simple string)

a = "my favorite color is blue"
b = "my favorite color is blue"
print(id(a) == id(b))    # False -- too long, not interned
```

2.4 Multiple Assignment

Python supports elegant multiple assignment using tuple packing/unpacking:

```
x, y = 6, 5           # Assign in one line
x, y = y, x           # Swap! Behind the scenes: tuple pack then unpack.
a = b = c = 0         # Chain assignment -- all point to same 0 object
```

3. Data Types

3.1 Numeric Types

```
x = 10            # int -- unlimited precision in Python
pi = 3.14         # float -- IEEE 754 double precision
c = 3 + 4j        # complex -- uses 'j' not 'i'

# Type conversion
k = int(pi)       # 3  -- truncates, doesn't round
p = float(k)      # 3.0
c = complex(2, 3)  # (2+3j)

# Check
print(type(x))  # <class 'int'>
```

Note

Boolean is a subtype of int: `True == 1` and `False == 0`. So `int(True)` gives 1. You can even do `True + True == 2`.

3.2 Strings

Strings are **immutable sequences of characters**. Once created, characters cannot be changed in-place.

```
name = 'Naven'           # single quotes
msg  = "Hello world"     # double quotes
para = """Line one
Line two"""              # triple quotes -- multi-line

# Indexing (0-based)
print(name[0])      # 'N'   (first character)
print(name[-1])     # 'n'   (last character)

# Slicing [start:stop:step]  -- stop is EXCLUSIVE
text = 'telusko'
print(text[3:7])    # 'usko'  -- indices 3,4,5,6
print(text[:3])     # 'tel'   -- from start to index 2
print(text[3:])     # 'usko'  -- from index 3 to end
print(text[-4:])    # 'usko'  -- last 4 characters
print(text[::2])    # 'tlso'  -- every second character

# Repetition
print('Na' * 3)     # 'NaNaNa'

# f-strings (preferred formatting)
age = 30
print(f"My name is {name} and I am {age} years old.")
print(f"Pi is approximately {3.14159:.2f}")  # 2 decimal places
```

3.2.1 Escape Sequences

Escape	Meaning
\n	Newline
\t	Tab
\\	Literal backslash
\'	Single quote inside single-quoted string
\"	Double quote inside double-quoted string

```
# Problem: Windows path
path = "C:\\Users\\Naven"    # double backslash to escape
# OR use raw string:
path = r"C:\Users\Naven"     # r prefix means no escape processing
```

3.2.2 Key String Methods

```
s = "  Hello, Python!  "
print(s.strip())          # "Hello, Python!"  -- remove whitespace
print(s.lower())          # "  hello, python!  "
print(s.upper())          # "  HELLO, PYTHON!  "
print(s.replace("Python", "World"))  # "  Hello, World!  "
print(s.split(","))       # ["  Hello", " Python!  "]
print(",".join(["a", "b", "c"]))  # "a,b,c"
print(s.find("Python"))   # index of first match (-1 if not found)
print(len(s))             # length including spaces
```

3.3 Lists

Lists are **ordered, mutable, allow duplicates** and can hold mixed types.

```
nums = [45, 87, 21, 24, 99]       # list of integers
mix  = [1, "hello", 3.14, True]   # mixed types -- totally fine

# Access
print(nums[0])      # 45
print(nums[-1])     # 99

# Slicing works same as strings
print(nums[1:3])    # [87, 21]

# Common methods
nums.append(33)     # Add at end
nums.insert(1, 55) # Insert 55 at index 1; shifts others right
nums.remove(87)     # Remove first occurrence of value 87
val = nums.pop()    # Remove and return LAST element
val = nums.pop(2)   # Remove and return element at index 2
nums.reverse()      # Reverse in-place
nums.sort()         # Sort in-place (ascending by default)
count = nums.count(21)    # Number of times 21 appears
idx   = nums.index(24)    # Index of first occurrence of 24

# del keyword for slices
```

```python
del nums[2:4]       # Remove elements at index 2 and 3

# Extend vs Append
nums.extend([100, 200])  # Adds each element from iterable
nums.append([100, 200])  # Adds the LIST as a single element

# Python built-in functions on lists
print(min(nums), max(nums), sum(nums), len(nums))
```

Note

List inside list (nested): `mix = [nums, names]` – each inner list is a single element. Access with `mix[0][2]` – first list, third element.

3.4 Tuples

Tuples are **ordered and immutable**. Use them when data should not change.

```python
t = (23, 45, 67, 43)    # with parentheses
t = 23, 45, 67, 43      # parentheses optional -- same result

# Single-element tuple NEEDS trailing comma
single = (42,)    # This is a tuple
not_tuple = (42) # This is just int 42 in parentheses

# Access and slicing work same as list
print(t[1])      # 45
print(t[-1])     # 43
print(t[1:3])    # (45, 67)

# Only two methods (can't mutate)
print(t.count(45))  # occurrences
print(t.index(67))  # index of value

# Immutable = cannot reassign elements
# t[0] = 99  -> TypeError: tuple does not support item assignment

# Tuple UNPACKING
num, name, score = (1, "Naven", 95.5)
print(num, name, score)

# Multiple return values use tuple implicitly
def minmax(lst):
    return min(lst), max(lst)    # returns a tuple

lo, hi = minmax([3, 1, 7, 2])
```

Tip

When to use tuple vs list? Tuple = fixed data (coordinates, RGB colour, database row). List = collection that grows/shrinks. Tuples are slightly faster and signal "this won't change".

3.5 Sets

Sets are **unordered collections of unique elements**. Perfect for membership testing and eliminating duplicates.

```
s1 = {23, 56, 78, 21, 56}  # duplicate 56 silently removed
print(s1)   # {56, 21, 78, 23} -- order may vary (hashed
    internally)

# Empty set MUST use set() -- {} creates an empty DICT
empty = set()

# Membership test -- O(1) speed due to hashing
print(21 in s1)   # True
print(99 in s1)   # False

# Set operations (mirrors math set theory)
s2 = {'a', 'b', 'c', 'd', 'm'}
s3 = {'a', 'e', 'i', 'o', 'u', 'd', 'p'}

print(s2 - s3)    # Difference: in s2 but NOT in s3
print(s2 | s3)    # Union: all elements from both
print(s2 & s3)    # Intersection: only common elements
print(s2 ^ s3)    # Symmetric difference: in one but NOT both
```

Note

Why unordered? Each element is hashed. Python uses the hash to decide where to store it – this makes lookup O(1) instead of O(n). The side effect: order is not preserved.

3.6 Dictionaries

Dictionaries are **key-value pairs**. Keys form a set (unique, unordered). Values form a list (can repeat). Think of a real dictionary: word -> meaning.

```
phone_book = {
    "Naven":  9876543210,
    "Kiran":  8765432109,
    "Hush":   7654321098,
}

# Access by key (KeyError if missing)
print(phone_book["Naven"])      # 9876543210

# .get() -- safe access, returns None (or default) if key missing
print(phone_book.get("Naven"))          # 9876543210
print(phone_book.get("Unknown"))        # None
print(phone_book.get("Unknown", -1))    # -1 (custom default)

# Add / update
phone_book["Social"] = 6543210987   # Add new key
phone_book["Naven"] = 1111111111    # Update existing key

# Delete
```

```python
phone_book.pop("Hush")        # Remove and return value
del phone_book["Social"]      # Remove without returning

# Iteration
for key in phone_book:
    print(key, "->", phone_book[key])

for key, val in phone_book.items():    # Pythonic way
    print(f"{key}: {val}")

# Nested dict / dict with list values
data = {
    "js":     "VSCode",
    "python": ["VSCode", "PyCharm"],
    "java":   {"core": "VSCode", "spring": "IntelliJ"}
}
print(data["java"]["spring"])    # "IntelliJ"
```

3.6.1 Creating from Sets + Lists

```python
keys   = {"Naven", "Hitesh", "Shamik"}  # set -- unique keys
values = [34, 64, 87]                   # list of values

# zip + dict -- combines into dict
d = dict(zip(keys, values))
# Note: set order is not guaranteed, so key-value pairing may vary
    .
# Use a list for keys if order matters:
keys = ["Naven", "Hitesh", "Shamik"]
d = dict(zip(keys, values))
print(d)    # {'Naven': 34, 'Hitesh': 64, 'Shamik': 87}
```

4. Operators

4.1 Arithmetic Operators

Operator	Name	Example
+	Addition	3 + 5 = 8
-	Subtraction	5 - 2 = 3
*	Multiplication	3 * 4 = 12
/	True Division	8 / 2 = 4.0 (always float)
//	Floor Division	9 // 2 = 4 (quotient)
%	Modulus	9 % 2 = 1 (remainder)
**	Exponentiation	2 ** 10 = 1024

Note

Python follows **BODMAS/PEMDAS**. Multiplication happens before addition. Use parentheses to override: `(9 + 2) * 3 = 33` vs `9 + 2 * 3 = 15`.

4.2 Comparison (Relational) Operators

These always return a `bool` (`True` or `False`).

```
a, b = 4, 5
print(a > b)     # False
print(a < b)     # True
print(a >= b)    # False
print(a <= b)    # False
print(a == b)    # False  -- double equals for comparison
print(a != b)    # True
```

Watch Out

`=` is *assignment*. `==` is *comparison*. Confusing them is the most common beginner error.

4.3 Logical Operators – Truth Table

C1	C2	C1 and C2	C1 or C2	not C1
True	True	True	True	False
True	False	**False**	True	False
False	True	**False**	True	True
False	False	**False**	**False**	True

```
a, b = 4, 5
print(a < 10 and b > 1)     # True AND True  -> True
print(a < 10 and b > 10)    # True AND False -> False (and needs
    both)
print(a < 10 or  b > 10)    # True OR  False -> True  (or needs
    just one)
print(not (a == b))          # not False -> True
```

4.4 Assignment Shorthand

```
a = 5
a += 2    # a = a + 2  ->  7
a -= 1    # a = a - 1  ->  6
a *= 3    # a = a * 3  ->  18
a //= 4   # a = a // 4 ->  4
a **= 2   # a = a ** 2 ->  16
a %= 5    # a = a % 5  ->  1
```

5. Control Flow

5.1 if / elif / else

Python uses **indentation** (not curly braces) to define blocks. This enforces readable code.

```
num = int(input("Enter a number: "))

if num % 2 == 0:
```

```
    print("Even")
elif num % 2 != 0:
    print("Odd")
else:
    print("This won't run")    # redundant here but shows else
        syntax
```

Tip

The `else` in an if/elif chain is optional. `elif` can appear as many times as needed. There can only be one `else`. As soon as a condition is `True`, the rest are skipped.

5.1.1 Nested if

```
num = int(input("Enter a number: "))
salary = int(input("Enter salary: "))

if num % 2 == 0:
    print("Even number")
    if salary >= 5:
        print("Great job!")
    else:
        print("Better luck next time.")
else:
    print("Odd number")
```

5.2 match Statement (Python 3.10+)

A cleaner alternative to long `elif` chains when matching specific values:

```
num = 3

match num:
    case 1: print("One")
    case 2: print("Two")
    case 3: print("Three")
    case 4: print("Four")
    case 5: print("Five")
    case _: print("Number out of range")    # _ is the default/
        wildcard
```

Note

Unlike C's `switch`, Python's `match` does NOT fall through. No `break` needed. The `_` case is the default – like `else`.

5.3 while Loop

Repeats a block as long as a condition is `True`.

```
i = 1
while i <= 5:
    print(f"Hello world {i}")
```

```
    i += 1              # CRITICAL: without this, infinite loop!
print(f"Final value of i: {i}")    # 6  (exits when i=6 fails the
    check)
```

5.3.1 Nested while – "Hour and Day" Analogy

```
i = 1
while i <= 3:           # outer loop (days)
    j = 1               # reset j for every outer iteration (day
        resets hours)
    while j <= 4:       # inner loop (hours)
        print("Telusko " * 1, end="")
        print("rocks " * 1, end="")
        j += 1
    print()             # newline after each outer iteration
    i += 1
```

Watch Out

Infinite loop trap: If you forget to initialize `j` inside the outer loop, after the first outer iteration `j` stays at 5, and the inner loop never executes again.

5.4 for Loop

The `for` loop iterates over any **iterable** (list, tuple, string, range, dict, etc.).

```
# Over a list
data = [2, "Naven", 4.5, 8, "Telusko", "Python"]
for value in data:
    print(value)

# Over a range
for i in range(10):           # 0, 1, 2, ..., 9
    print(i)

for i in range(2, 11, 2):    # even numbers: 2, 4, 6, 8, 10
    print(i)

# Over dict (keys by default)
for key in phone_book:
    print(key, "->", phone_book[key])

# Over string (character by character)
for ch in "Python":
    print(ch)
```

5.5 break and continue

```
# continue -- skip current iteration
for i in range(10):
    if i % 3 == 0:
        continue        # skip multiples of 3
```

```
    print(i)            # prints 1, 2, 4, 5, 7, 8

# break -- exit loop entirely
for i in range(10):
    if i == 5:
        break           # stop at 5
    print(i)            # prints 0, 1, 2, 3, 4

print("bye")            # always prints (outside loop)
```

Tip

for...else and while...else: Python has an optional `else` clause on loops. The `else` block runs if the loop completed *without* hitting a `break`.

6. Functions

6.1 Why Functions?

Functions let you **write code once, call it many times**. They also give a name to a block of logic, making code readable. Python's built-ins like `print()`, `len()`, `int()` are all functions defined elsewhere.

```
# Without function -- repetitive
a, b = 4, 5
c = a + b
print(c)
# ... 50 lines later, same logic again
a, b = 10, 20
c = a + b
print(c)

# With function -- DRY (Don't Repeat Yourself)
def add(x, y):
    """Adds two values and returns the result."""
    return x + y

result = add(4, 5)
print(result)         # 9
result = add(10, 20)
print(result)         # 30
```

Note

The triple-quoted string immediately after `def` is a **docstring**. It documents what the function does. Access it with `add.__doc__` or hover in your IDE.

6.2 Parameters and Arguments

6.2.1 Default Arguments

```
def greet(name, greeting="Hello"):
```

```
    print(f"{greeting}, {name}!")

greet("Naven")                # Hello, Naven!   (uses default)
greet("Naven", "Namaste")     # Namaste, Naven!  (overrides default)
```

6.2.2 Keyword Arguments

When calling a function, you can specify arguments by name – order doesn't matter.

```
def person(name, age=18):
    print(f"Name: {name}, Age: {age}")

person(age=30, name="Naven")  # keyword args -- order flexible
```

6.2.3 Variable-Length Positional Arguments (*args)

```
def add(first, *rest):
    """first is a regular arg. rest is a tuple of extra args."""
    total = first
    for n in rest:
        total += n
    return total

print(add(4, 5, 6, 7))    # 22  -- works for any number of extra
   args
```

6.2.4 Variable-Length Keyword Arguments (**kwargs)

```
def person(name, **extra_info):
    """extra_info is a dict of any keyword args passed."""
    print(f"Name: {name}")
    for key, val in extra_info.items():
        print(f"  {key}: {val}")

person("Naven", age=30, location="Pune", tech="Python")
# Name: Naven
#   age: 30
#   location: Pune
#   tech: Python
```

6.3 Return Values and None

```
def add(a, b):
    return a + b           # explicit return value

def greet(name):
    print(f"Hello {name}")  # no return -- implicitly returns None

result = greet("Naven")
print(result)               # None

# Functions can return multiple values (as a tuple)
```

```python
def divmod_manual(a, b):
    return a // b, a % b    # quotient and remainder

q, r = divmod_manual(9, 2)
print(q, r)    # 4  1
```

6.4 Scope: Global vs Local Variables

```python
a = 10    # global variable

def something():
    a = 15    # This creates a NEW local variable 'a' -- does NOT
        change global!
    print(f"Inside function: {a}")    # 15

something()
print(f"Outside function: {a}")    # 10 -- global unchanged

# To modify the global variable inside a function:
def change_global():
    global a
    a = 20    # Now this modifies the global 'a'

change_global()
print(a)    # 20

# globals() -- returns a dict of all current global variables
print(globals()['a'])    # 20
```

Warning

Using `global` is generally considered bad practice in large codebases. Prefer passing values as arguments and returning them. Reserve `global` for truly necessary cases.

7. Functional Programming Tools

7.1 Functions as First-Class Objects

In Python, functions are objects. They can be:

- Assigned to variables
- Passed as arguments to other functions
- Returned from functions
- Stored in lists/dicts

This is the foundation of functional programming in Python.

7.2 Lambda (Anonymous Functions)

A `lambda` is a one-liner function – no name, no `def`, no `return` statement needed.

```python
# Regular function
```

```
def square(n): return n * n

# Lambda equivalent
square = lambda n: n * n

# Multi-parameter lambda
add = lambda a, b: a + b
print(add(3, 4))    # 7

# Used directly in expressions
print((lambda x: x ** 2)(5))    # 25
```

Tip

Use lambdas for short, one-off functions, especially inside `map()`, `filter()`, `reduce()`, or when passing a function to another function.

7.3 map, filter, reduce

```
from functools import reduce

nums = [4, 2, 9, 7, 6, 8, 3, 1]

# filter -- keep elements where function returns True
evens = list(filter(lambda n: n % 2 == 0, nums))
print(evens)    # [4, 2, 6, 8]

# map -- transform each element
doubles = list(map(lambda n: n * 2, evens))
print(doubles)    # [8, 4, 12, 16]

# reduce -- fold all elements into one value
total = reduce(lambda a, b: a + b, doubles)
print(total)    # 40

# Combining in one pipeline:
result = reduce(
    lambda a, b: a + b,
    map(lambda n: n * 2,
        filter(lambda n: n % 2 == 0, nums))
)
print(result)    # 40
```

7.4 Higher-Order Functions

A function that **takes or returns another function** is called a higher-order function.

```
def square(n):  return n * n
def cube(n):    return n * n * n

def operate(nums, operation):
    """Higher-order function -- accepts any single-arg function."""
```

```
        "
    for n in nums:
        result = operation(n)
        print(f"{operation.__name__}({n}) = {result}")

operate([5, 6, 7], square)
operate([5, 6, 7], cube)
operate([5, 6, 7], lambda n: n + 100)    # anonymous inline
```

7.5 Inner Functions and Closures

```
def outer():
    def inner(x):
        print(f"Inner says: {x}")
    inner(42)           # can call inner from within outer
    return inner        # return the function OBJECT (not call it)

fn = outer()    # fn is now the inner function
fn(99)          # calls inner(99) -- Inner says: 99

# Closure -- inner function captures outer scope
def counter_factory(start):
    count = start
    def increment():
        nonlocal count    # nonlocal lets inner modify outer's var
        count += 1
        return count
    return increment

counter = counter_factory(10)
print(counter())    # 11
print(counter())    # 12
print(counter())    # 13
```

7.6 Decorators

A decorator is a function that **wraps another function** to add behavior before/after it, without modifying the original function.

```
def log_deco(func):
    """Decorator that logs function calls."""
    def wrap(*args, **kwargs):
        print(f"Calling '{func.__name__}' with args={args}")
        result = func(*args, **kwargs)
        print(f"Result: {result}")
        return result
    return wrap

@log_deco           # Syntactic sugar: divide = log_deco(divide)
def divide(a, b):
    return a / b

```

```
divide(10, 2)
# Calling 'divide' with args=(10, 2)
# Result: 5.0
```

```
# Decorator that enforces "greater number first" behavior
def greater_first(func):
    def wrap(a, b):
        if a < b:
            a, b = b, a      # swap so larger is always first
        return func(a, b)
    return wrap

@greater_first
def divide(a, b):
    return a / b

@greater_first
def subtract(a, b):
    return a - b

print(divide(2, 8))     # 8/2 = 4.0  (not 2/8!)
print(subtract(3, 9))   # 9-3 = 6
```

Note

Why decorators matter in real projects: Frameworks like Flask/Django use decorators heavily – `@app.route('/home')` tells Flask which function handles the `/home` URL. `@login_required` protects routes. You write the logic; the decorator adds infrastructure.

8. Modules, Packages & __name__

8.1 Modules

A **module** is any `.py` file. When it contains only function/class definitions (no top-level execution), it acts as a library for other files.

```
# calc.py (a module)
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

```
# demo.py (uses the module)
import calc                          # import whole module
print(calc.add(3, 4))                # 7 -- must prefix with module
    name

from calc import add                 # import specific function
print(add(3, 4))                     # 7 -- no prefix needed

```

```
from calc import add, subtract    # import multiple
from calc import *                # import everything (avoid --
   pollutes namespace)
import calc as c                  # alias
print(c.add(3, 4))
```

8.2 Packages

A **package** is a folder of modules. Group related modules into packages.

```
my_project/
  calculator/          <- package (folder)
    calc.py            <- module
    demo.py            <- module
  main.py              <- entry point
```

```
# main.py -- accessing from outside the package
from calculator import calc      # or
from calculator.calc import add
```

8.3 The __name__ Variable

```
# When you RUN a file directly:     __name__ == '__main__'
# When a file is IMPORTED:          __name__ == 'module_name'

# calc.py
def add(a, b):
    return a + b

if __name__ == '__main__':
    # This block only runs when calc.py is run directly
    # NOT when calc.py is imported by demo.py
    print(add(2, 3))    # test code
```

Tip

Always put test/demo code inside `if __name__ == '__main__':`. This way, your file works both as a module (importable) and as a standalone script.

8.4 The math Module – Example

```
import math

print(math.sqrt(25))      # 5.0
print(math.ceil(59.4))    # 60  (ceiling -- always rounds UP)
print(math.floor(59.9))   # 59  (floor  -- always rounds DOWN)
print(math.pow(5, 3))     # 125.0 (5 to the power 3)
print(math.pi)            # 3.141592653589793

# help(math) or help(math.sqrt) shows docstrings in REPL
```

9. Object-Oriented Programming (OOP)

9.1 The OOP Mindset

OOP models software around **objects** rather than functions. Every real-world thing has:

- **Properties** (what it *knows*) – stored as variables
- **Behaviour** (what it *does*) – stored as methods (functions inside class)

Class = Blueprint. Object = Instance created from that blueprint.

One blueprint -> many objects, just as one building plan -> many identical buildings.

9.2 Defining a Class and Creating Objects

```
class Computer:
    brand = "Telusko"        # CLASS VARIABLE -- shared by all
        instances

    def __init__(self, cpu, ram, ssd):
        # INSTANCE VARIABLES -- unique per object
        self.cpu = cpu
        self.ram = ram
        self.ssd = ssd

    def config(self):
        """Print configuration of this computer."""
        print(f"Config: {self.cpu}, {self.ram} RAM, {self.ssd} SSD
            ")

# Create objects (instances)
comp1 = Computer("i5",  "16GB", "512GB")
comp2 = Computer("i9",  "96GB", "2TB")

comp1.config()   # Config: i5,  16GB RAM, 512GB SSD
comp2.config()   # Config: i9,  96GB RAM,   2TB SSD

# Class variable accessed via class name or instance
print(Computer.brand)   # Telusko
print(comp1.brand)      # Telusko
```

9.2.1 What is self?

`self` is the first parameter of every instance method. It is a reference to *the object calling the method*. Python passes it automatically – you never pass `self` explicitly when calling.

```
comp1.config()
# Python internally translates this to:
Computer.config(comp1)   # comp1 becomes 'self' inside the method
```

You can name it anything, but `self` is universally agreed upon. Use it.

9.2.2 Three Types of Methods

```
class Computer:
```

```
    brand = "Telusko"

    def __init__(self, cpu, ram):
        self.cpu = cpu
        self.ram = ram

    # 1. Instance method -- works with instance variables via self
    def config(self):
        print(f"{self.cpu}, {self.ram}")

    # 2. Class method -- works with class variables via cls
    @classmethod
    def info(cls):
        print(f"Brand: {cls.brand}")

    # 3. Static method -- utility method, no self or cls
    @staticmethod
    def gb_to_bytes(gb):
        return gb * (1024 ** 3)

c = Computer("i7", "32GB")
c.config()                # instance method
Computer.info()           # class method
Computer.gb_to_bytes(16) # static method
```

9.2.3 Constructor vs __init__

```
# __new__ is the true constructor -- creates the object
# __init__ is the initializer -- sets up the object's state
# Python calls __new__ then __init__ for every `ClassName()` call.
# You almost never need to define __new__ unless implementing
   singletons.

class A:
    def __new__(cls):
        print("Constructor called -- object being created")
        return super().__new__(cls)   # must return the object

    def __init__(self):
        print("Initializer called -- setting up state")

a = A()
# Constructor called -- object being created
# Initializer called -- setting up state
```

9.3 Inheritance

Inheritance lets one class **reuse** the variables and methods of another.

```
class A:
    def feature1(self):  print("F1 works")
    def feature2(self):  print("F2 works")

```

```
class B(A):          # B inherits from A -- single inheritance
    def feature3(self):  print("F3 works")
    def feature4(self):  print("F4 works")

class C(B):          # C inherits from B which inherits from A --
   multi-level
    def feature5(self):  print("F5 works")

obj = C()
obj.feature1()  # C -> B -> A -- found in A
obj.feature5()  # found in C itself
```

9.3.1 Multiple Inheritance

```
class A:
    def show(self): print("in A show")

class B:
    def show(self): print("in B show")

class C(A, B):     # inherits from both A and B
    pass

c = C()
c.show()    # "in A show" -- MRO: C -> A -> B (left to right)

# To explicitly call B's show:
B.show(c)    # "in B show"
```

9.3.2 Method Resolution Order (MRO)

```
print(C.__mro__)
# (<class 'C'>, <class 'A'>, <class 'B'>, <class 'object'>)
# Python searches left to right, then 'object' (root of all
   classes)
```

9.3.3 super() and __init__ Chains

```
class A:
    def __init__(self):
        print("A init")
    def feature1(self): print("F1 works")

class B(A):
    def __init__(self):
        super().__init__()    # call A's __init__
        print("B init")
    def feature3(self): print("F3 works")

obj = B()
# A init
# B init
```

```
15
16 obj.feature1()  # from A (inherited)
17 obj.feature3()  # from B
```

9.4 Polymorphism

"One interface, many forms." The same operation behaves differently depending on the object.

9.4.1 Duck Typing

```
1  class Laptop:
2      def build(self):
3          print("Laptop builds")
4
5  class Desktop:
6      def build(self):
7          print("Desktop builds")
8
9  class Tablet:
10     def open_pdf(self):        # NO build() method!
11         print("Tablet opens PDF")
12
13 def developer_works(machine):
14     machine.build()            # just needs a .build() method
15
16 developer_works(Laptop())     # works
17 developer_works(Desktop())    # works -- different class, same
       interface
18 developer_works(Tablet())     # AttributeError -- no .build()!
```

Note

Duck typing: "If it quacks like a duck, it is a duck." Python doesn't check the type – it just calls the method. If the method exists, it works.

9.4.2 Operator Overloading

You can define what `+`, `>`, `str()` etc. mean for your custom class by implementing dunder (magic) methods.

```
1  class Account:
2      def __init__(self, name, balance):
3          self.name    = name
4          self.balance = balance
5
6      def __str__(self):
7          """Called by print() / str()."""
8          return f"{self.name}: {self.balance}"
9
10     def __add__(self, other):
11         """Defines what '+' means for two Account objects."""
12         return Account("Combined", self.balance + other.balance)
13
14     def __gt__(self, other):
```

```
        """Defines what '>' means."""
        return self.balance > other.balance

u1 = Account("Naven", 1000)
u2 = Account("Kiran", 2000)

print(u1)             # Naven: 1000  (uses __str__)
combined = u1 + u2    # uses __add__
print(combined)       # Combined: 3000
print(u1 > u2)        # False  (1000 > 2000)  (uses __gt__)
```

9.4.3 Method Overriding

When a child class redefines a parent method with the same name, the child's version is used.

```
class A:
    def show(self): print("in A show")

class B(A):
    def show(self): print("in B show")    # OVERRIDES A's show

b = B()
b.show()    # "in B show" -- child's version wins
```

9.5 Abstraction

Abstraction means **hiding implementation details** and exposing only what's necessary. In Python, this is achieved through abstract classes (ABC).

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    """Interface -- defines what every gateway MUST have."""

    @abstractmethod
    def pay(self, amount):
        pass    # no implementation -- just the contract

class RazorPay(PaymentGateway):
    def pay(self, amount):
        print(f"Paying Rs. {amount} via RazorPay")

class Telusko Pay(PaymentGateway):
    def pay(self, amount):
        print(f"Paying Rs. {amount} via TeluskoP ay")

# Can NOT instantiate abstract class:
# gw = PaymentGateway()  -> TypeError: Can't instantiate abstract
   class

rp = RazorPay()
rp.pay(500)    # Paying Rs. 500 via RazorPay
```

Tip

Real-world use: Your `checkout` page depends on `PaymentGateway`, not `RazorPay` specifically. Switch to a different gateway by just passing a different object – no changes to checkout code. This is **loosely coupled design**.

10. Recursion

10.1 What is Recursion?

A function calling itself. Every recursion needs:

1. A **base case** – the stopping condition (or it runs forever)
2. A **recursive case** – reducing the problem toward the base case

```
# Factorial: n! = n * (n-1) * ... * 1
def factorial(n):
    if n == 1:            # base case
        return 1
    return n * factorial(n - 1)    # recursive call

print(factorial(5))  # 120
# Call stack: factorial(5) -> 5 * factorial(4)
#             factorial(4) -> 4 * factorial(3)  ... -> factorial(1) = 1
# Unwinds:    1 -> 2 -> 6 -> 24 -> 120
```

```
import sys

# Default recursion limit
print(sys.getrecursionlimit())    # 1000

# Change it (use carefully)
sys.setrecursionlimit(3000)
```

Watch Out

Each recursive call adds a frame to the call stack. Too many recursive calls -> **RecursionError: maximum recursion depth exceeded**. For large inputs, prefer an iterative solution.

11. Exception Handling

11.1 Types of Errors

Type	When	Example
Syntax Error	At parse time	`deff foo():`
Logical Error	Wrong output, no crash	Returning 2 instead of 1 in recursio
Runtime Error / Exception	At runtime, unexpected input/state	Divide by zero

11.2 Exception Hierarchy

```
BaseException
  +-- Exception
        +-- ArithmeticError
        |     +-- ZeroDivisionError
        +-- ValueError
        +-- TypeError
        +-- FileNotFoundError
        +-- IndexError
        +-- KeyError
        +-- RecursionError
        +-- ... (many more)
```

11.3 try / except / else / finally

```python
a = int(input("Numerator: "))
b = int(input("Denominator: "))

try:
    result = a / b
    print(f"Result: {result}")
except ZeroDivisionError as e:
    print(f"Error: Cannot divide by zero. ({e})")
except ValueError as e:
    print(f"Error: Invalid input -- please enter numbers only. ({e})")
except Exception as e:
    print(f"Unexpected error: {e}")
else:
    print("Calculation successful!")  # runs only if NO exception
finally:
    print("End of execution.")    # ALWAYS runs -- clean up resources here
```

Note

The `finally` block: Always executes, whether or not an exception occurred. Perfect for closing files, database connections, or network sockets – you must close resources even if your code crashes.

11.4 Raising Exceptions Manually

```python
def set_age(age):
    if age < 0 or age > 150:
        raise ValueError(f"Invalid age: {age}. Must be 0-150.")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e)    # Invalid age: -5. Must be 0-150.
```

12. Concurrency: Threads & Multiprocessing

12.1 Why Concurrency?

Modern applications do many things at once: download files, play music, update UI, respond to user clicks. Without concurrency, each task would block the others.

- **Thread:** Lightweight unit of execution. Multiple threads share the same process memory.
- **Process:** Heavier. Has its own memory space and Python interpreter (own GIL).

12.2 Threads with `threading` Module

```python
import threading
import time

def download(filename):
    print(f"Downloading {filename}...")
    time.sleep(0.5)   # simulate download time
    print(f"Downloaded {filename}!")

files = ["video.mp4", "image.png", "data.csv"]

# Serial (no threads)
start = time.time()
for f in files:
    download(f)
print(f"Serial: {time.time() - start:.2f}s")  # ~1.5s

# Parallel (with threads)
threads = []
start = time.time()
for f in files:
    t = threading.Thread(target=download, args=(f,))
    threads.append(t)
    t.start()            # LAUNCH each thread

for t in threads:
    t.join()             # WAIT for all threads to finish

print(f"Parallel: {time.time() - start:.2f}s")  # ~0.5s
```

```python
# Class-based approach (inheriting Thread)
class Hello(threading.Thread):
    def run(self):           # must be named 'run'
        for i in range(5):
            print(f"Hello {i+1}")

class Hi(threading.Thread):
    def run(self):
        for i in range(5):
            print(f"Hi {i+1}")

t1, t2 = Hello(), Hi()
```

```
t1.start()
t2.start()
t1.join()
t2.join()
print("bye")
```

12.3 The GIL Problem

Python's **Global Interpreter Lock (GIL)** allows only one thread to execute Python bytecode at a time. This means:

Task Type	Threads help?	Why
I/O-bound (file, network)	**Yes**	GIL released during I/O waits
CPU-bound (heavy math)	**No**	GIL prevents parallel CPU use

12.4 Multiprocessing for CPU-bound Tasks

```
from multiprocessing import Process
import time

def calculate(n1, n2):
    total = sum(i ** 2 for i in range(n1, n2))

num = 50_000_000
mid = num // 2

# Each process has its OWN GIL -- true parallelism
p1 = Process(target=calculate, args=(0, mid))
p2 = Process(target=calculate, args=(mid, num))

start = time.time()
p1.start(); p2.start()
p1.join();  p2.join()
print(f"Multiprocessing: {time.time() - start:.2f}s")
# Noticeably faster than serial for CPU-bound work
```

Tip

Use `threading` for I/O-bound tasks. Use `multiprocessing` for CPU-bound tasks. For truly async I/O at scale (web servers), look into `asyncio`.

13. Arrays (array module)

```
from array import array

# array(typecode, [values]) -- homogeneous typed array
arr = array('i', [10, 20, 30, 40, 50])    # 'i' = signed int

print(arr)          # array('i', [10, 20, 30, 40, 50])
print(list(arr))    # [10, 20, 30, 40, 50]  -- convert to list to
   print cleanly
```

```
8
9  # Operations
10 arr.append(60)                # add at end
11 arr.insert(1, 15)             # insert 15 at index 1
12 arr.remove(30)                # remove first occurrence of 30
13 val = arr.pop()               # pop last element
14 print(arr.typecode)           # 'i'
15 print(arr.buffer_info())      # (address, length)
16
17 # Copy an array efficiently (generator expression)
18 arr2 = array(arr.typecode, (n for n in arr))
19
20 # Iterate
21 for n in arr:
22     print(n)
```

Note

array vs list: Arrays require all elements to be the same type – faster for numeric operations and uses less memory. For complex array work (2D arrays, matrix ops), use NumPy instead.

14. Quick Reference – Patterns & Idioms

14.1 List Comprehensions

```
1 # [expression for item in iterable if condition]
2 squares = [x**2 for x in range(10)]
3 evens   = [x for x in range(20) if x % 2 == 0]
4 matrix  = [[i*j for j in range(1,4)] for i in range(1,4)]
```

14.2 Dictionary / Set Comprehensions

```
1 word_lengths = {word: len(word) for word in ["hello", "world", "
     python"]}
2 # {'hello': 5, 'world': 5, 'python': 6}
3
4 unique_squares = {x**2 for x in range(-5, 6)}
5 # {0, 1, 4, 9, 16, 25}
```

14.3 enumerate & zip

```
1 names = ["Naven", "Kiran", "Hush"]
2
3 for i, name in enumerate(names, start=1):
4     print(f"{i}. {name}")    # 1. Naven, 2. Kiran, 3. Hush
5
6 scores = [95, 87, 72]
7 for name, score in zip(names, scores):
8     print(f"{name}: {score}")
```

14.4 Walrus Operator := (Python 3.8+)

```
import re
data = "Hello 42 World"

if m := re.search(r'\d+', data):
    print(f"Found number: {m.group()}")    # Found number: 42
```

14.5 Context Managers (with)

```
# Automatically closes the file even if an exception occurs
with open("data.txt", "r") as f:
    content = f.read()
# file is closed here -- no need for explicit f.close()
```

14.6 Unpacking & Starred Assignment

```
first, *middle, last = [1, 2, 3, 4, 5]
print(first)    # 1
print(middle)   # [2, 3, 4]
print(last)     # 5
```

15. Complete OOP Example – Putting It All Together

```
from abc import ABC, abstractmethod

# ----------- ABSTRACT BASE CLASS (Interface) -----------
class Shape(ABC):
    @abstractmethod
    def area(self): pass

    @abstractmethod
    def perimeter(self): pass

    def __str__(self):
        return f"{self.__class__.__name__}: area={self.area():.2f}
           "

# ----------- CONCRETE CLASSES (Implementations) -----------
class Circle(Shape):
    PI = 3.14159

    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return Circle.PI * self.radius ** 2

    def perimeter(self):
        return 2 * Circle.PI * self.radius
```

```
26
27 class Rectangle(Shape):
28     def __init__(self, width, height):
29         self.width  = width
30         self.height = height
31
32     def area(self):
33         return self.width * self.height
34
35     def perimeter(self):
36         return 2 * (self.width + self.height)
37
38 # ----------- POLYMORPHISM IN ACTION -----------
39 shapes = [Circle(5), Rectangle(4, 6), Circle(3)]
40
41 for shape in shapes:
42     print(shape)                # uses __str__
43     print(f"  Perimeter: {shape.perimeter():.2f}")
44
45 # All shapes respond to area() and perimeter() -- duck typing!
46 total_area = sum(s.area() for s in shapes)
47 print(f"Total area: {total_area:.2f}")
```

16. Mental Models – Summary

Section Summary

- **Variables = labels, not boxes.** They point to objects. Objects have identity (`id`), type (`type`), value.
- **Everything is an object.** Integers, strings, functions, classes – all objects.
- **Mutable vs Immutable.** Lists/dicts/sets are mutable. Strings/tuples/ints are immutable.
- **self = current instance.** Passed automatically. Write it, don't call it.
- **Inheritance chains.** MRO is left-to-right, depth-first. Use `super()` to call parent.
- **Duck typing.** Python doesn't care about the class, only whether the method exists.
- **Decorators = wrappers.** `@dec` above `def f` means `f = dec(f)` behind the scenes.
- **GIL.** Threads for I/O. Processes for CPU. `asyncio` for high-concurrency I/O.
- **Exceptions.** `try/except/finally` – never let the app crash on predictable errors.
- **Modules = .py files. Packages = folders of modules.** `__name__ == '__main__'` guards execution.

Python Notes – Intermediate-level reference. Pair with code practice for best retention.